

Reading Your Output

When you run the auto-grader, you'll see a block of output for each sorting algorithm that ran. This handout explains what every column and bar means so you can read your results — and your classmates' — without guessing.

The Output, At a Glance

Here's what one section looks like for an in-place sort (bubble) on the 100,000-element data set:

```
= ----- =
= BUBBLE SORT          GRADE    HH:MM:SS:mmm    INPUT          AUX =
= ----- =
= ELEMENTS:    100,000  SORTING
=                PASSED    00:00:43:218    781.3K    0.00B =
Live[                                0.00B/783K]
Peak[                                0.00B/783K]
Tot [                                0.00B/12.2M]
```

And here's the same section for merge sort on the same data set:

```
= ----- =
= MERGE SORT          GRADE    HH:MM:SS:mmm    INPUT          AUX =
= ----- =
= ELEMENTS:    100,000  SORTING
=                PASSED    00:00:00:089    781.3K    781.3K =
Live[                                0.00B/783K]
Peak[|||||                                781K/783K]
Tot [|||||                                12.0M/12.2M]
```

There are two parts to read: the bordered result row in the middle (with `PASSED`, time, `INPUT`, `AUX`) and the three memory bars below it (Live, Peak, Tot). The two parts measure different things and answer different questions.

The Result Row

This row reports the headline numbers for the run:

```
=                PASSED    00:00:00:089    781.3K    781.3K =
```

Column	What it tells you
GRADE	Either <code>PASSED</code> or the program aborts. The grader runs your sort, then independently sorts the same input with the system <code>qsort</code> , and compares element-by-element. If your output matches, you pass; if any element disagrees, the program prints the mismatch and aborts.
HH:MM:SS:mmm	Wall-clock time the sort took, in hours, minutes, seconds, and milliseconds. This is real elapsed time on the machine, not CPU cycles or operation count.
INPUT	The actual heap-allocated bytes for your input array. For 100,000

Column	What it tells you
	long long integers (each 8 bytes) the program asks for 800,000 bytes; the allocator typically gives you slightly more due to alignment and per-block overhead. The number reported here is what was really carved out of the heap, queried via <code>malloc_usable_size</code> .
AUX	Peak auxiliary memory your sort allocated during execution, on top of the input array. This is the column that distinguishes algorithms — see the next section.

Reading the AUX Column

AUX is the most pedagogically interesting number on the row. It answers the question: "How much extra memory did this algorithm need beyond the input array itself?"

What you should see

- **Bubble Sort:** ≈ 0 bytes (you may see ~ 16 B from framework overhead — see note below)
- **Insertion Sort:** ≈ 0 bytes (same as bubble — both are in-place)
- **Merge Sort:** $\approx n \times \text{sizeof}(T)$ — for 100,000 long long integers, that's 800,000 bytes $\approx 781.3\text{K}$

If your bubble or insertion AUX shows hundreds of bytes or more, your sort is allocating something it doesn't need to — most likely a temporary array or a `std::vector` inside a loop. The whole point of these sorts is that they sort *in place*, meaning they swap elements within the array you were given without ever asking the allocator for more space.

Note about the 16-byte baseline: The auto-grader uses a background thread to display the live memory bars, and starting that thread allocates a small (~ 16 byte) bookkeeping object. That allocation happens *before* your sort runs and is unavoidable, so it shows up as a tiny floor on the AUX value even for in-place sorts. Anything close to 16 B means "my sort allocated nothing" — the framework is what you're seeing.

The Three Memory Bars

Below the result row, three bars show different views of your sort's heap usage in real time. All three numbers come from a tracker built into the auto-grader that intercepts every C++ heap allocation (every new and every vector growth) and counts the bytes.

Column	What it tells you
Live	Bytes <i>currently outstanding</i> on the heap. Goes up when you allocate, down when you free. For an in-place sort it stays at zero. For merge sort it dances up and down as L and R buffers are allocated and freed at each level of the recursion. After the sort returns, Live drops back to 0 because everything has been freed.
Peak	The <i>high-water mark</i> — the highest value Live ever reached during the sort. This is what you compare across algorithms. For

Column	What it tells you
	bubble/insertion, Peak stays at 0; for merge, Peak ends at $n \times \text{sizeof}(T)$ (the size of the topmost merge's L plus R).
Tot	<i>Cumulative</i> bytes allocated since the sort started. Every <code>new</code> adds to this number; freeing does <i>not</i> subtract from it. For merge sort this hits roughly $n \times \log_2(n) \times \text{sizeof}(T)$ — much larger than Peak, because each level of the recursion allocates fresh L and R buffers, even though the previous level's buffers have already been freed.

How the bars are scaled

The Live and Peak bars are scaled to your input size — a fully-filled bar means the algorithm allocated as much auxiliary memory as the input array itself. The Tot bar is scaled to $16 \times \text{input size}$, which fits comfortably for typical sorts (merge's $n \log n$ cumulative work fills it about 80% on the 100K data set). For very large inputs the Tot bar may saturate at 100% — that itself is informative; it means "this sort did a lot of allocation work."

The text after each bar shows the actual values: `current/scale`. For example, `Peak[|||| 781K/783K]` means "peak heap usage was 781K out of an 783K reference scale," which the bar visualizes as roughly 99% filled.

What the Numbers Tell You

The point of running all three sorts on the same data set is to see, side-by-side, the memory and time trade-offs that show up in lecture as Big-O notation. The auto-grader makes those trade-offs measurable:

In-place sorts (Bubble, Insertion)

- $AUX \approx 0$ (framework overhead only)
- All three bars stay flat at 0%
- Time: $O(n^2)$ — slow on large inputs
- Pedagogical takeaway: "in-place" really does mean zero extra memory

Out-of-place sort (Merge)

- $AUX \approx n \times \text{sizeof}(T)$ — proportional to input size
- Live bar dances during the sort, Peak fills, Tot grows monotonically
- Time: $O(n \log n)$ — much faster on large inputs
- Pedagogical takeaway: merge sort trades memory for speed

Comparing the trade-off

On the 1,000,000-element data set, bubble sort takes roughly half an hour while merge sort finishes in under a second. That ratio is the practical consequence of the difference between n^2 and $n \log n$. The cost merge sort pays for that speed is exactly what the AUX column shows: about 8 megabytes of extra memory it had to ask the allocator for. On modern hardware, 8 MB is a fair price for cutting half an hour down to a fraction of a second — which is why production sorting libraries are almost never bubble sort.

Common Questions

Why does my Live bar look empty after the sort?

Because at that moment it really is empty — the sort returned, and every allocation it made has been destroyed. Live measures "right now," not "during." The Peak bar is what retains the visible record of the high-water mark, which is why Peak and Tot stay filled while Live drops to zero on the final frame.

Why is the Tot bar bigger than the Peak bar?

Peak is the most memory live at *any single moment* during the sort. Tot is the *sum* of every allocation, even ones that have already been freed. Merge sort allocates and frees over and over again at each recursion level — Peak only sees the largest snapshot, while Tot sums them all. For n elements, $\text{Peak} \approx n$ bytes but $\text{Tot} \approx n \times \log_2(n)$ bytes.

Why does my AUX show some non-zero bytes for an in-place sort?

If it's roughly 16 bytes, that's the framework's monitoring thread setup (see the note in the AUX section above). If it's hundreds or thousands of bytes, your sort is allocating something. Common culprits: declaring a `std::vector`, `std::string`, or any object with internal heap storage inside the sort function — even if you only use it briefly, the allocation still gets counted. Move such variables outside the sort, or use stack arrays / pointers into the input array instead.

Why does merge sort run so much faster than bubble on big inputs but they look similar on small ones?

On small inputs, the difference between n^2 and $n \log n$ is small in absolute terms — for $n=100$, the gap is 10,000 vs 700 operations, which both finish in microseconds. On larger inputs, the gap widens dramatically: at $n=1,000,000$, n^2 is a trillion operations and $n \log n$ is twenty million. The auto-grader's data sets range from 10 to 1,000,000 elements specifically so you can watch this gap open up.

Submitting Your Final Grade

When you're ready to submit your final grade, run the auto-grader and choose option 4) `Run all (final grade submission)` from the algorithm menu. This runs all three sorts on the data set you selected and prints the full memory and timing report for each. Take a screenshot of the entire output and submit that to Canvas as your final.